

Abhishek Raghav

Btech CSE

CU24250156

Experiment No.7: Motion Estimation Using Optical Flow Algorithm in Videos Sequences

Aim

To implement optical flow algorithms for estimating the motion of objects between consecutive video frames and analyze motion patterns for dynamic scene understanding using Python and OpenCV.

Objectives

1. To understand the concept of optical flow.
2. To implement Lucas–Kanade Sparse Optical Flow.
3. To implement Farneback Dense Optical Flow.
4. To visualize motion vectors and trajectories.
5. To compare sparse and dense optical-flow techniques.
6. To study the effect of object speed, lighting, and camera motion.

✓ Theory

1. Optical Flow

Optical Flow is the apparent motion of objects, surfaces, or camera movement between consecutive frames of a video.

It represents the motion of pixels using vectors. Each vector contains:

- **Direction:** The direction in which the pixel/object moves.
- **Magnitude:** The amount/speed of movement.

Optical flow is useful in various applications, including:

- Object tracking
- Autonomous driving
- Video surveillance
- Robotics

- Gesture recognition
- Sports analytics
- Augmented reality

These applications are also highlighted in the supplied experiment description.

2. Lucas–Kanade Optical Flow

Lucas–Kanade is a **Sparse Optical Flow** algorithm. Instead of calculating motion for every pixel, it tracks selected feature points between consecutive frames.

Working Principle:

The algorithm is based on the brightness constancy assumption:

$$I(x, y, t) = I(x + u, y + v, t + 1)$$

where:

- (I) = image intensity
- (u) = horizontal motion
- (v) = vertical motion

Using image gradients, this can be approximated as:

$$I_x u + I_y v + I_t = 0$$

Lucas–Kanade assumes that neighboring pixels have approximately the same motion and solves the resulting equations using a least squares method.

Advantages:

- Computationally efficient
- Good for tracking selected features
- Suitable for real-time applications

Disadvantages:

- Tracks only selected points
- Depends on good feature points
- Can struggle with large or very fast motion

3. Farneback Optical Flow

Farneback is a **Dense Optical Flow** algorithm. It estimates motion for almost every pixel in an image rather than only selected feature points.

The resulting flow field contains:

- Horizontal displacement

- Vertical displacement
- Direction
- Magnitude

The experiment specifically requires displaying the dense flow using color-coded visualization.

Difference Between Sparse and Dense Optical Flow

Feature	Sparse Optical Flow	Dense Optical Flow
Example	Lucas–Kanade	Farneback
Points calculated	Selected feature points	Almost every pixel
Computational cost	Lower	Higher
Output	Motion vectors of features	Complete motion field
Speed	Faster	Relatively slower
Applications	Object/feature tracking	Motion analysis, segmentation
Visualization	Arrows/trajectories	Color-coded flow

Part A — Lucas–Kanade Sparse Optical Flow

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display, Image, clear_output
import time # To control display rate

# Load video
cap = cv2.VideoCapture("/content/vecteezy_drifting-car-.mov")

# Read first frame
ret, old_frame = cap.read()

if not ret:
    print("Could not read video")
    cap.release()
    # Exit or return is not ideal in a Colab cell, print and proceed with clear

# Convert first frame to grayscale
old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

# Detect good feature points
feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)
```

```

p0 = cv2.goodFeaturesToTrack(old_gray, mask=None, **feature_params)

# Parameters for Lucas-Kanade
lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(
        cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT,
        10,
        0.03
    )
)

mask = np.zeros_like(old_frame)
frame_count = 0
display_interval = 5 # Display every 5th frame
max_frames_to_display = 100 # Limit the total number of frames to display

while True:
    ret, frame = cap.read()

    if not ret:
        break

    frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    p1, st, err = cv2.calcOpticalFlowPyrLK(
        old_gray,
        frame_gray,
        p0,
        None,
        **lk_params
    )

    if p1 is not None and len(p1) > 0:
        # Select successfully tracked points
        good_new = p1[st == 1]
        good_old = p0[st == 1]

        # If no points are successfully tracked, re-detect
        if len(good_new) == 0:
            p0 = cv2.goodFeaturesToTrack(frame_gray, mask=None, **feature_params)
            mask = np.zeros_like(frame) # Reset mask for new features
            if p0 is None or len(p0) == 0:
                print("No features to track. Breaking loop.")
                break
            continue # Skip drawing for this frame, process next frame with new

        # Draw motion trajectories
        for new, old in zip(good_new, good_old):

```

```

a, b = new.ravel()
c, d = old.ravel()
a, b, c, d = map(int, [a, b, c, d])

mask = cv2.line(
    mask,
    (a, b),
    (c, d),
    (255, 255, 255),
    2
)
frame = cv2.circle(
    frame,
    (a, b),
    5,
    (255, 255, 255),
    -1
)

output = cv2.add(frame, mask)

# Update points for the next iteration
p0 = good_new.reshape(-1, 1, 2)
else:
    # If p1 is None or empty (no points tracked), re-detect features for the
    p0 = cv2.goodFeaturesToTrack(frame_gray, mask=None, **feature_params)
    mask = np.zeros_like(frame) # Reset mask for new features
    if p0 is None or len(p0) == 0:
        print("No features to track. Breaking loop.")
        break
    output = frame # Display original frame if no features are being tracked

# Display frame in Colab
if frame_count % display_interval == 0:
    clear_output(wait=True)
    rgb_output = cv2.cvtColor(output, cv2.COLOR_BGR2RGB)
    _, encoded_img = cv2.imencode('.jpeg', rgb_output)
    display(Image(data=encoded_img.tobytes()))
    time.sleep(0.05) # Add a small delay to make the video playable

old_gray = frame_gray.copy()
frame_count += 1

if frame_count >= max_frames_to_display:
    print(f"Reached maximum display frames ({max_frames_to_display}). Stopping")
    break

cap.release()
# cv2.destroyAllWindows() # Not needed in Colab for this display method.

```



Reached maximum display frames (100). Stopping.

This implements the experiment's required process of reading consecutive frames, converting them to grayscale, tracking selected feature points, and visualizing their

motion using trajectories

Part B — Farneback Dense Optical Flow

```
import cv2
import numpy as np
from IPython.display import display, Image, clear_output
import time # To control display rate

cap = cv2.VideoCapture("/content/vecteezy_drifting-car-.mov")

ret, frame1 = cap.read()

if not ret:
    print("Could not read video")
    cap.release()
    # Exit or return is not ideal in a Colab cell, print and proceed with clear

prev_gray = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)

frame_count = 0
display_interval = 5 # Display every 5th frame
max_frames_to_display = 100 # Limit the total number of frames to display

while True:

    ret, frame2 = cap.read()

    if not ret:
        break

    next_gray = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

    # Calculate dense optical flow
    flow = cv2.calcOpticalFlowFarneback(
        prev_gray,
        next_gray,
        None,
        0.5,
        3,
        15,
        3,
        5,
        1.2,
        0
    )

    # Convert flow into magnitude and angle
    magnitude, angle = cv2.cartToPolar(
```

```

        flow[..., 0],
        flow[..., 1]
    )

    # HSV image for visualization
    hsv = np.zeros_like(frame2)

    hsv[..., 1] = 255

    # Direction represented by hue
    hsv[..., 0] = angle * 180 / np.pi / 2

    # Magnitude represented by brightness
    hsv[..., 2] = cv2.normalize(
        magnitude,
        None,
        0,
        255,
        cv2.NORM_MINMAX
    )

    # Convert HSV to BGR
    optical_flow = cv2.cvtColor(
        hsv,
        cv2.COLOR_HSV2BGR
    )

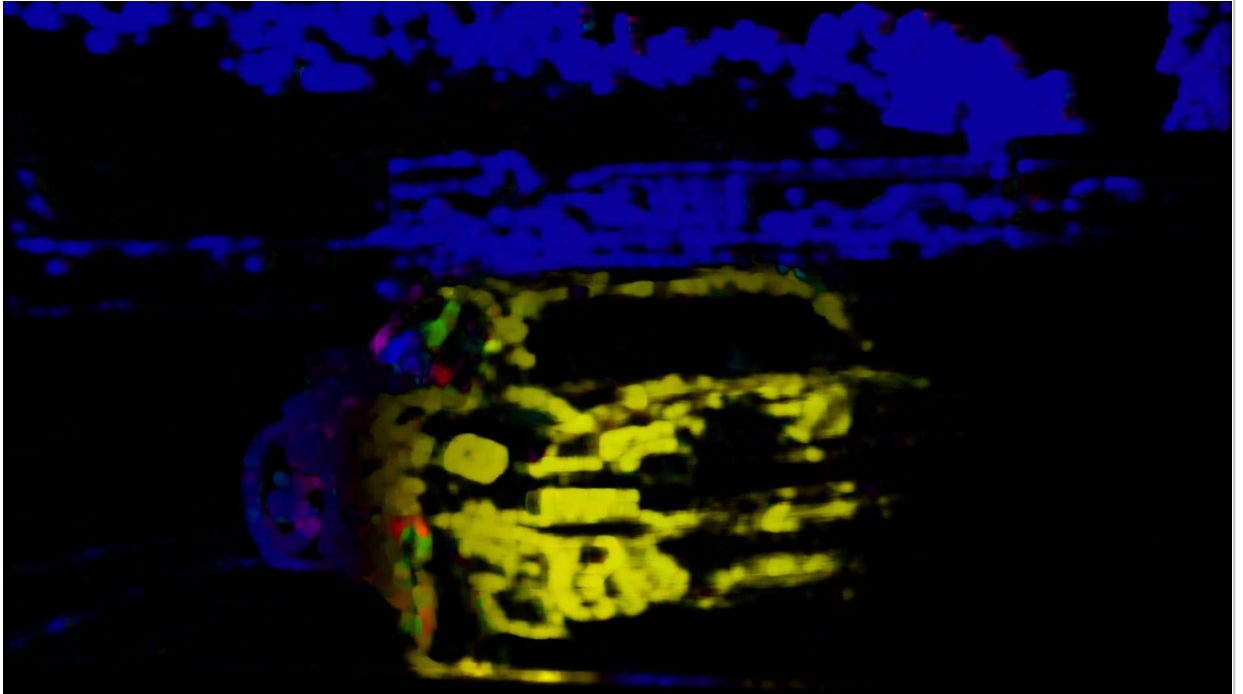
    # Display frame in Colab
    if frame_count % display_interval == 0:
        clear_output(wait=True)
        rgb_optical_flow = cv2.cvtColor(optical_flow, cv2.COLOR_BGR2RGB)
        _, encoded_img = cv2.imencode('.jpeg', rgb_optical_flow)
        display(Image(data=encoded_img.tobytes()))
        time.sleep(0.05) # Add a small delay to make the video playable

    prev_gray = next_gray
    frame_count += 1

    if frame_count >= max_frames_to_display:
        print(f"Reached maximum display frames ({max_frames_to_display}). Stop")
        break

cap.release()
# cv2.destroyAllWindows() # Not needed in Colab for this display method.

```



Reached maximum display frames (100). Stopping.

Farneback estimates motion over the image and uses direction and magnitude to create a color-coded representation, as required by the lab sheet.

Algorithm

Lucas–Kanade

- Read the video.
- Read two consecutive frames.
- Convert frames to grayscale.
- Detect good feature points.
- Track these points in the next frame.
- Calculate their displacement.
- Draw arrows/trajectories.
- Display the result.
- Repeat for subsequent frames.

Farneback

- Read the video.
- Read consecutive frames.
- Convert them into grayscale.
- Calculate dense optical flow.
- Calculate magnitude and direction.
- Convert the flow information into HSV representation.
- Convert HSV to BGR.
- Display the color-coded motion.
- Repeat for subsequent frames.

Observations

Observation	Result
Stationary object	Very small/no motion vector
Slowly moving object	Small motion vectors
Fast-moving object	Larger motion vectors
Camera movement	Motion appears across a large portion of the frame
Good lighting	Better feature tracking
Poor lighting	Reduced accuracy
Lucas–Kanade	Tracks selected feature points
Farneback	Produces dense motion information

Comparison

Parameter	Lucas–Kanade	Farneback
Type	Sparse	Dense
Motion estimation	Selected points	Nearly every pixel
Speed	Fast	Comparatively slower
Computational requirement	Low	High
Output	Point trajectories	Dense flow field
Feature requirement	Requires features	Does not require explicit feature selection
Suitable for	Tracking	Detailed motion analysis
Real-time suitability	Excellent	Moderate
Motion visualization	Arrows/paths	Color map

Applications

Five major applications of optical flow are:

- **Autonomous driving** – detecting movement of vehicles, pedestrians, and surrounding objects.
- **Video surveillance** – detecting and tracking moving objects.
- **Robotics** – estimating environmental and camera motion.
- **Gesture recognition** – analyzing hand and body movement.
- **Sports analytics** – analyzing player and ball movement.

The supplied experiment also identifies autonomous driving, surveillance, robotics, gesture recognition, sports analytics, and augmented reality as applications.

Results

Lucas–Kanade Sparse Optical Flow successfully estimates the motion of selected feature points between consecutive video frames and represents the movement through trajectories.

Farneback Dense Optical Flow successfully estimates motion over the image and represents the direction and magnitude of motion using a color-coded flow visualization.

Thus, both algorithms can be used for motion estimation, with Lucas–Kanade being more suitable for efficient feature tracking and Farneback being more suitable when dense motion information is required.

Conclusion

In this experiment, optical flow was studied and implemented using Lucas–Kanade Sparse Optical Flow and Farneback Dense Optical Flow. Lucas–Kanade tracks selected feature points and is computationally efficient, whereas Farneback estimates motion over a dense set of pixels and provides detailed motion information. The performance of both methods depends on factors such as object speed, lighting conditions, image quality, and camera motion. Optical flow is therefore an important technique for dynamic scene analysis and motion-based computer vision systems.

Viva Questions & Answers

1. What is Optical Flow?

Optical flow is the apparent motion of pixels or objects between consecutive frames of a video. It is used to estimate object and camera motion.

2. Explain Lucas–Kanade Optical Flow.

Lucas–Kanade is a sparse optical-flow method that tracks selected feature points between consecutive frames. It assumes that nearby pixels have similar motion and calculates their displacement.

3. What is Dense Optical Flow?

Dense optical flow estimates motion for almost every pixel in an image, unlike sparse optical flow, which estimates motion only for selected feature points.

4. Compare Lucas–Kanade and Farneback.

Lucas–Kanade is sparse and computationally efficient, while Farneback is dense and computationally more expensive but provides motion information across the image.

5. What assumptions are made in optical flow?

Common assumptions include:

- Brightness constancy
- Small motion between consecutive frames
- Spatial consistency/similar motion of neighboring pixels

6. What factors affect optical-flow accuracy?

Important factors include:

- Lighting changes
- Object speed
- Camera movement
- Motion blur
- Occlusion
- Low-texture regions
- Noise
- Large displacement

7. Give five applications.

- Autonomous driving
- Video surveillance
- Robotics
- Gesture recognition
- Sports analytics

8. Why are grayscale images used?

Grayscale reduces the image from three color channels to one intensity channel, reducing computational complexity while retaining the intensity information needed for gradient and motion calculations.

9. What are the limitations?

Optical-flow algorithms can be affected by illumination changes, occlusions, motion blur, large displacement, textureless regions, and camera motion.

10. How does optical flow help autonomous driving and surveillance?

It helps estimate the movement of vehicles, pedestrians, and other objects. This information can be used for tracking, collision prediction, activity detection, and scene understanding. These types of applications are explicitly included in the experiment's evaluation objectives